AcroSplit: Mass-Balanced Red/Green Watermarking for LLM-Generated Code

Anonymous ACL submission

Abstract

We describe AcroSplit, a multi-bit watermark for language-model code generation. After ordinary sampling filters, every step with at least two surviving candidates is split into green and red lists of similar probability mass; the next message bit selects which list to sample from. A generation-time W-score measures how strongly those splits constrain the sample. On Qwen3.8-27B, a short list-conversion function carries 20 embedding steps at 0.22 bits per token in executable code, with no comment padding. Average density is 0.16–0.18 bits/token, about three times ACW and SrcMarker, with 100% bit accuracy. Relative to classic (unwatermarked) sampling, pass@1 drops 2.1% on InstructHumanEval and 2.8% on MBPP. On a vLLM serving stack the same protocol holds for smaller code models: Qwen3-8B loses 2.2% relative pass@1 on HumanEval, and DeepSeek-Coder-6.7B-Instruct loses 3.1%.

1 How AcroSplit works

AcroSplit hides a bitstring in the tokens chosen by a language model. Generation writes the message. Reveal reads it back. Both procedures share three inputs: the model, the decoding filters, and a secret key.

The message m has length L. Cue i writes bit $m_{i \bmod L}$. After the last bit, the message repeats from the start. Figure 1 shows one step of each procedure. Algorithms 1–3 state the same steps in full.

1.1 Generation

Start from the prompt. Then, at every step:

1. **Filter.** Apply the decoding filters. Keep the surviving candidates in a set C. Later draws stay inside C.
2. **Skip.** If C has fewer than two tokens, sample from C in the usual way. This step writes no bit.
3. **Split.** If C has two or more tokens, split it into two lists, L_0 and L_1. The rule is Section 1.2.
4. **Select.** Let b be the current message bit. Keep list L_b. Drop the other list.
5. **Sample.** Draw the next token from L_b. Inside that list, probabilities stay proportional to the filtered ones.
6. **Advance.** Append the token. The next cue carries the next message bit.

1.2 Split

Split is fixed by C, the key, and $\mathrm{context}(x)$: a short suffix of the prefix already written. $\mathrm{mass}(L)$ is the total filtered probability of the tokens in L. L_0 is the list for bit 0. L_1 is the list for bit 1.

1. Sort the tokens of C from the most probable to the least probable.
2. Give each token, in that order, to the list whose mass is smaller.
3. If the two masses are equal, hash the key together with $\mathrm{context}(x)$, and use that bit to choose the list. The first token is always a tie.

1.3 Reveal

Walk the generated tokens, starting again from the prompt.

1. Recompute C with the same filters on the same prefix.
2. If C has at least two tokens, recompute the same split, and advance the cue index.
3. The list that contains the observed token is the bit recorded for this cue.

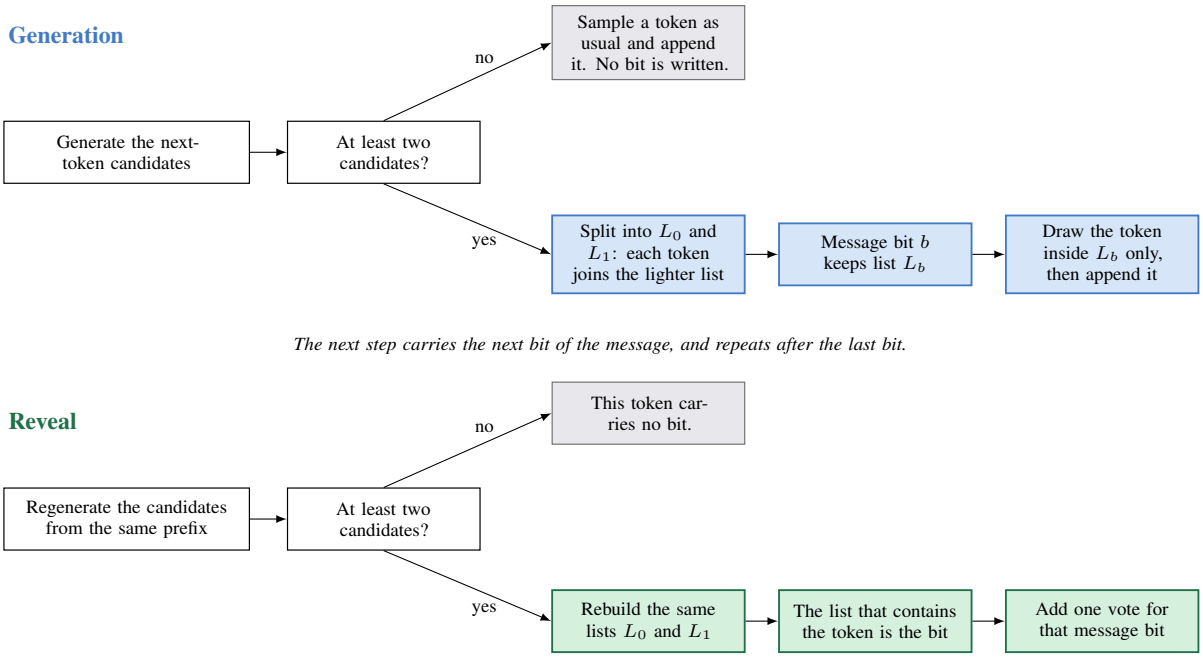


Figure 1: One generation step and the matching reveal step. Gray: the step has fewer than two candidates, so no bit is written or read. Blue: generation writes the current message bit by sampling inside one list. Green: reveal reads that bit back from the list which contains the observed token. L_0 is the list for bit 0. L_1 is the list for bit 1.

4. For each message index, count the votes for 0 and the votes for 1.

5. Write 1 when the votes for 1 are strictly more numerous. Write 0 in every other case, including a tie.

Algorithm 1. Split(C, key, x)

sort C by decreasing probability
$L_0 \leftarrow \emptyset$, $L_1 \leftarrow \emptyset$
$h \leftarrow \text{Hash}(\text{key}, \text{context}(x)) \bmod 2$
for each token t in C **do**
  if $\text{mass}(L_0) < \text{mass}(L_1)$ **then** $L_0 \leftarrow L_0 \cup \{t\}$
  else if $\text{mass}(L_1) < \text{mass}(L_0)$ **then** $L_1 \leftarrow L_1 \cup \{t\}$
  else $L_h \leftarrow L_h \cup \{t\}$
return (L_0, L_1)

Algorithm 2. Generate

$x \leftarrow$ prompt, $i \leftarrow 0$
while not done **do**
  $C \leftarrow \text{Filter}\big(p(\cdot \mid x)\big)$
  if $|C| < 2$ **then**
    $t \leftarrow \text{Sample}(C)$
  else
    $(L_0, L_1) \leftarrow \text{Split}(C, \text{key}, x)$
    $b \leftarrow m_{i \bmod L}$
    $t \leftarrow \text{Sample}(L_b)$ *(relative probabilities)*
    $i \leftarrow i + 1$
  append t to x
return the tokens appended to the prompt

Algorithm 3. Reveal

$\text{votes}_0[0..L-1] \leftarrow 0$, $\text{votes}_1[0..L-1] \leftarrow 0$
$x \leftarrow$ prompt, $i \leftarrow 0$
for each generated token t **do**
  $C \leftarrow \text{Filter}\big(p(\cdot \mid x)\big)$
  if $|C| \geq 2$ **then**
    $(L_0, L_1) \leftarrow \text{Split}(C, \text{key}, x)$
    $j \leftarrow i \bmod L$
    if $t \in L_0$ **then** $\text{votes}_0[j] \leftarrow \text{votes}_0[j] + 1$
    else if $t \in L_1$ **then** $\text{votes}_1[j] \leftarrow \text{votes}_1[j] + 1$
    $i \leftarrow i + 1$
  append t to x
for $j = 0$ **to** $L - 1$ **do**
  $\hat{m}_j \leftarrow 1$ **if** $\text{votes}_1[j] > \text{votes}_0[j]$ **else** 0
return $\hat{m}$

2 Related work

Work on marking machine-generated text has two largely separate lineages: *logit-level* watermarks that intervene while the model samples, and *post-hoc* watermarks that rewrite a finished artifact. AcroSplit belongs to the first lineage, but it is built for code, where a usable watermark has to carry a multi-bit payload inside short, executable programs rather than a single detect/not-detect bit in long prose.

The canonical logit-level scheme of Kirchenbauer et al. (2023) hashes the preceding tokens and partitions the *entire* vocabulary into green and red lists. Sampling is biased toward green; a z-test on

green hits then detects the watermark. That design is strong for long natural-language passages, and later variants make the bias unbiased or distortion-free (Zhao et al., 2023; Kuditipudi et al., 2024) or argue cryptographic undetectability (Christ et al., 2024). It is a poor fit for code generation on three counts. First, a random split of the full vocabulary ignores the tiny nucleus that nucleus sampling actually uses, so most of the green list is never a legal next token. Second, the usual objective is binary detection, not a recoverable message of a few bits. Third, a heavy green bias can change identifiers and control flow enough to fail unit tests. AcroSplit keeps the red/green vocabulary of that literature, but it splits only the filtered candidate set C, balances the two lists by probability mass rather than by a hash of token ids, and spends each cue on the next bit of a repeating payload. The mass balance is why either bit remains a high-probability continuation, which is the quantity that pass@1 measures.

A second line of work watermarks *source code after it has been written*. ACW (Li et al., 2024) and SrcMarker (Yang et al., 2024) encode bits by applying semantics-preserving transforms and identifier rewrites to a finished program. Those methods do not need the generator, so they apply to human code as well as to model output. The channel, however, is sparse: on HumanEval and MBPP snippets the static capacity is about one bit every 16–18 tokens (Section 4.2), and recovery is imperfect (97.2% and 97.3% BitAcc) because a transform that already matches the original is scored as an error. AcroSplit cannot mark a program that was not sampled under the key, but it does not wait for a finished AST either. Every splittable decoding step is a cue, density is about three times higher on Qwen3.8-27B, and reveal replays the same keyed split, so cue-level BitAcc is 100% whenever the split ran.

The evaluation of code generators themselves (Chen et al., 2021; Austin et al., 2021) is the right testbed for this distinction. A watermark that survives only as comments, or that drops pass@1 by a large relative margin, is not a watermark on *code generation*. Section 4 therefore reports functional pass@1 against matched classic sampling, not only retrieval.

3 Watermarking score

The lists hold only candidates that survived filtering, and the split keeps their masses close. A random green/red hash partitions the full vocabulary, with no mass balance. The two lists are rarely perfectly balanced. Write m_t for the total filtered probability of the list selected at cue t, measured *before* masking. A bit that picks the heavy side yields $m_t \gg 0.5$; a bit that picks the light side yields a small m_t. Unwatermarked text tends to occupy the heavier list (occupied mass around 0.6–0.63 in our checks). A watermarked sample, because the bit chooses independently of mass, sits closer to one half on average.

Hide always forces a hit on the chosen list, so a raw green-hit count is not informative at generation time. We instead accumulate a soft z-score from the chosen masses alone:

$$W = \frac{\sum_t (1 - m_t)}{\sqrt{\sum_t m_t\,(1 - m_t)}}. \qquad (1)$$

Each cue behaves like a Bernoulli trial that always succeeds while the null success probability is m_t. Sampling the light list (small m_t) contributes more; many cues near $m_t \approx 0.5$ still drive W upward. A large positive W on a generated sample means the sequence was produced under the watermark constraint over many unbalanced splits. It is a generation-time strength signal, not a stand-alone keyless detector.

4 Results

The following measurements share the same protocol: matched classic sampling versus AcroSplit, same prompts, same filters. We report an example, token density and BitAcc against post-hoc code watermarks, generation throughput, then pass@1 on Qwen3.8-27B and on additional models served with vLLM.

4.1 Hiding bits in short programs

Because every splittable nucleus step carries a bit, short functions can still host many cues even when the model writes ordinary executable statements rather than comments. Figure 2 shows MBPP task 117, `list_to_float`: recursively (classic) or by a list comprehension (AcroSplit), convert nested string fields to floats. Both completions are comment-free. AcroSplit uses 20 embedding steps (density 0.22) and recovers the 4-bit payload `0100` in full. The two programs differ in identifiers, control structure, and `str` vs. `repr`—the kind of variation the mass-balanced split is meant to harvest.

```python
def list_to_float(lst):
  def convert_to_float(item):
    if isinstance(item, (list, tuple)):
      return type(item)(convert_to_float(
        subitem) for subitem in item)
    try:
      return float(item)
    except (ValueError, TypeError):
      return item
  converted = convert_to_float(lst)
  return str(converted)
```

Original code

```python
def list_to_float(lst):
  def convert_to_float(item):
    try:
      return float(item)
    except (ValueError, TypeError):
      return item
  result = [
    tuple(convert_to_float(x) for x in inner)
    if isinstance(inner, (tuple, list))
    else convert_to_float(inner)
    for inner in lst
  ]
  return repr(result)
```

Watermarked code

Figure 2: Same MBPP task, both programs pass the unit tests. Left: classic sampling (original). Right: AcroSplit (watermarked; 20 cues). The 4-bit message is retrieved by majority vote over those cues.

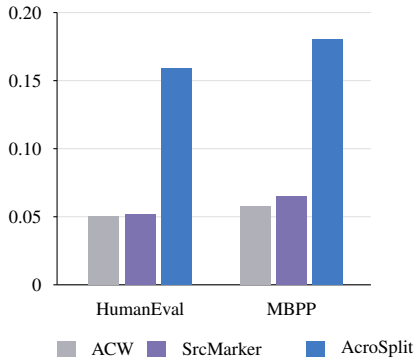


Figure 3: Bits per token.

	BitAcc (%)
ACW	97.2
SrcMarker	97.3
AcroSplit	100

Table 1: BitAcc. ACW: mean over 4-bit encodings, uncorrected (Li et al., 2024). SrcMarker: $SrcMarker_{\mathrm{TE}}$ on CSN-Java (Yang et al., 2024). AcroSplit: cue-level recovery on Qwen3.8-27B.

	Classic	AcroSplit
Tokens/s	2.381	2.370

Table 2: Generation throughput (tokens/s).

4.2 Token density

Token density is hideable bits per token. ACW (Li et al., 2024) and SrcMarker (Yang et al., 2024) post-process a finished program: each applicable transform (or name rewrite) is one bit. On 539 HumanEval and MBPP snippets that yields 0.056 and 0.061 bits/token—about one bit every 16–18 tokens. AcroSplit instead spends a bit at every splittable decoding step. On Qwen3.8-27B that is 0.159 bits/token on InstructHumanEval and 0.180 on MBPP (Figure 3), roughly 3× denser.

Bit accuracy (BitAcc) is the fraction of embedded bits recovered in the clear. ACW checks whether each transform was applied; a transform that already matches by chance is an error, so BitAcc averages 97.2% (Li et al., 2024). SrcMarker learns which transforms and names to use; BitAcc is 97.3% on Java (Yang et al., 2024). AcroSplit reveal replays the same keyed split. The replay is deterministic, so the votes for one message index agree, and every cue is recovered: BitAcc is 100%.

4.3 Generation throughput

AcroSplit is applied after the filtered next-token distribution is already known. The extra work is a partition of that shortlist and a draw from one side. We measure end-to-end speed in tokens per second.

Classic sampling produces 2.381 tokens/s and AcroSplit 2.370 tokens/s (Table 2). The two rates match.

4.4 Pass@1 versus classic sampling

Table 3 compares functional pass@1 of classic sampling against AcroSplit. The runs share the prompts and the decoding filters: temperature, top-k, and nucleus. Because the split balances probability mass, either message bit stays a plausible continuation under the model. Relative change is $(w-u)/u$ against the matched classic run.

On Qwen3.8-27B, AcroSplit loses 1.8 points (2.1% relative) on InstructHumanEval and 2.0 points (2.8% relative) on full MBPP. Most classic-passing tasks still pass with the watermark. Density and cue-level BitAcc on that model are those of

Section 4.2.

4.5 Additional models on vLLM

The Qwen3.8-27B numbers use an OpenAI-compatible completions API that returns the top-20 log-probabilities at every step; AcroSplit then splits that shortlist on the client. The same protocol runs on other vLLM endpoints without changing the split.

Qwen3-8B, HumanEval. We generate classic and AcroSplit completions for all 164 HumanEval tasks (T=0.9, nucleus 0.9, top-20). Classic pass@1 is 82.3% and AcroSplit 80.5% (relative $\Delta = -2.2\%$; Table 3). Mean density is 0.145 bits/token. Reveal replays the same keyed split, so BitAcc is 100%. The drop in pass@1 is a utility cost of the split, not a failure of reveal.

DeepSeek-Coder-6.7B-Instruct, HumanEval. The same HumanEval suite on a vLLM DeepSeek-Coder-6.7B-Instruct endpoint yields classic pass@1 67.1% and AcroSplit 65.0% (relative $\Delta = -3.1\%$), density 0.138 bits/token, and BitAcc 100%.

In both cases the extra decision work after the top-20 list is already available remains sub-millisecond, consistent with Section 4.3: serving, not the split, dominates wall time.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Miranda Christ, Sam Gunn, and Or Zamir. 2024. Undetectable watermarks for language models. In *Proceedings of Thirty Seventh Conference on Learning Theory*, pages 1125–1139.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 17061–17084.

Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2024. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research*.

Boquan Li, Mengdi Zhang, Peixin Zhang, Jun Sun, Xingmei Wang, Zijian Liu, and Tianzi Zhang. 2024. Resilient watermarking for AI-generated codes. *arXiv preprint arXiv:2402.07518*.

Borui Yang, Wei Li, Li Xiang, and Bo Li. 2024. SrcMarker: Dual-channel source code watermarking via scalable code transformations. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE.

Xuandong Zhao, Prabhanjan Ananth, Lei Li, and Yu-Xiang Wang. 2023. Provable robust watermarking for AI-generated text. *arXiv preprint arXiv:2306.17439*.

Model	Dataset	Classic	AcroSplit	Rel. Δ	Dens.	BitAcc
Qwen3.8-27B	InstructHumanEval	86.6	84.8	-2.1%	0.159	100
Qwen3.8-27B	MBPP	71.0	69.0	-2.8%	0.180	100
Qwen3-8B	HumanEval	82.3	80.5	-2.2%	0.145	100
DeepSeek-Coder-6.7B	HumanEval	67.1	65.0	-3.1%	0.138	100

Table 3: Pass@1 (%), mean density (bits/token), and cue-level BitAcc (%). InstructHumanEval and HumanEval n=164; MBPP is the full test set. BitAcc is 100% because reveal replays the same keyed split. Rel. Δ is $(w-u)/u$ vs. classic sampling.